

UNIDAD 10

PROGRAMACIÓN FUNCIONAL Y LENGUAJES MULTIPARADIGMA

Programación
CFGS DAW

Autor: José Miguel Blázquez
2021/2022

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

 Importante

 Atención

 Interesante

ÍNDICE DE CONTENIDO

1. Introducción.....	4
2. Programación funcional.....	4
2.1 Declarativo vs imperativo: ejemplos.....	5
2.2 Características de la programación funcional.....	6
2.3 Expresiones lambda.....	7
2.4 Programación declarativa en Java: uso de Streams.....	8
3. Lenguajes multiparadigma y programación con Python.....	10
3.1 Características de otros lenguajes multiparadigma con enfoque web.....	10
3.2 El tipado de datos.....	11
3.3 El lenguaje de programación Python.....	13
3.3.1 Características de Python.....	13
3.3.2 Variables y tipos de datos.....	14
3.3.3 Variables numéricas.....	14
3.3.4 Cadenas de caracteres (string).....	15
3.3.5 Entrada y salida básica.....	15
3.3.6 Estructuras de control (condicionales y bucles).....	15
3.3.7 Estructuras de datos.....	17

UD10. PROGRAMACIÓN FUNCIONAL Y LENGUAJES MULTIPARADIGMA

1. INTRODUCCIÓN

En esta unidad vamos a conocer con un poco de detalle un estilo de programación del paradigma declarativo que, actualmente, está ganando popularidad en ámbitos donde la Programación Orientada a Objetos era la reina indiscutible. Sin embargo, y por el momento, no está llamada a sustituirla sino más bien a complementarla; nos referimos a la programación funcional, la cual está siendo añadida a lenguajes tradicionalmente imperativos u orientados a objetos como Java, JavaScript, C#...

En la siguiente parte de la unidad describiremos los lenguajes multiparadigma y aprenderemos a programar en uno de los lenguajes de mayor proyección hoy en día, y uno de los más versátiles, exponente claro del concepto de lenguaje multiparadigma actual, y ese es Python.

2. PROGRAMACIÓN FUNCIONAL

La Programación Funcional es un “paradigma de programación”, concretamente un tipo específico de la programación declarativa. Como ya estudiamos a principio de curso, un paradigma es una forma de pensar y de organizar un programa basado en una serie de principios (que detallaremos más adelante). Como comparación, el paradigma predominante estudiado hasta ahora era la Programación Orientada a Objetos, junto a la programación imperativa y la estructurada.

La programación funcional se basa fuertemente en el concepto de función matemática y otras propiedades. Ésta tiende a resultar en código más conciso y expresivo; se presta para expresar y resolver problemas actuales de paralelización, computación distribuida, asincronía, eventos, ... en el ámbito de aplicaciones web podemos citar a Whatsapp y Facebook, por ejemplo. En el primero se utiliza Erlang, un lenguaje de programación funcional; y en el segundo se usa Haskell para el sistema antispam. Incluso en bases de datos ya que SQL es declarativo.

Actualmente contamos con una gran cantidad de lenguajes en donde podemos hacer uso de la programación funcional, por ejemplo :

- Java
- PHP
- Ruby
- Python

- C#
- Kotlin

Muchos de estos lenguajes no están 100% enfocados en la programación funcional, lo cual no es malo, de hecho, estos nos permite hacer una transición de un paradigma a otro “en cualquier momento”, comúnmente de la programación orientada a objetos a la programación funcional.

Cabe resaltar que la programación funcional conlleva e incluye muchísimos más conceptos de los que vamos a tratar aquí, pero en esta unidad nos centraremos en lo más importante y utilizable en la práctica, aunque de forma resumida.

2.1 Declarativo vs imperativo: ejemplos

A diferencia de la programación estructurada y la POO (programación orientada a objetos), que son paradigmas “imperativos”, la programación funcional es “declarativa”.

Esto quiere decir que en el estilo funcional expresamos nuestra lógica sin describir el “control de flujo” (bucles, condicionales, ...), enfocándonos en “qué” estamos haciendo en vez de “cómo” se está haciendo, que sería el enfoque imperativo.

Como ejemplo comparativo, el lenguaje SQL también es declarativo.

Veamos un ejemplo

La problemática planteada es conocer la cantidad de elementos en esta lista mayores a 10.

```
List<Integer> numeros = List.of(18, 6, 4, 15, 55, 78, 12, 9, 8);
```

Podemos resolver este problema utilizando bucles y condicionales: enfoque **imperativo**.

```
int contador = 0;
for(int numero : numeros) {
    if(numero > 10) {
        contador++;
    }
}
System.out.println(contador);
```

Ahora vamos a verlo mediante el enfoque **declarativo**:

```
long result = numeros.stream().filter(num -> num > 10).count();
System.out.println(result);
```

Cómo observamos, con el enfoque declarativo las líneas de código se reducen y el código se torna más legible. Se puede observar cómo primero se filtra por una condición y posteriormente se suma lo filtrado.

Esta última implementación podemos concluir que es funcional, ya que delega el control

de flujos y condiciones a funciones (*filter* y *sum*).

2.2 Características de la programación funcional

Al trabajar con programación funcional, nos toparemos con conceptos tales como:

- Funciones puras.
- Composición de funciones.
- Estados compartidos.
- Mutabilidad.
- Efecto secundario.

Veamos cada uno de estos conceptos con mayor detalle:

FUNCIONES PURAS

Las funciones puras no son más que funciones las cuales, dando la misma entrada, siempre devuelven la misma salida, además de no tener efectos secundarios. Se dice que son inmutables y deterministas.

En la programación funcional no hay bucles `for` y `while`. En cambio, los lenguajes de programación funcionales dependen de la recursividad para la iteración.

COMPOSICIÓN DE FUNCIONES

La composición de funciones es el proceso de combinar dos o más funciones teniendo como finalidad ejecutar cada una de estas en secuencia (o en cadena) para obtener un resultado en concreto.

```
> f(x)
> g(x)
> f(g(x))
```

En este caso combinamos la función `f` (de `x`) con la función `g` (de `x`).

ESTADO COMPARTIDO

El estado compartido es cualquier variable, objeto o espacio de memoria que exista en un ámbito compartido. Un ámbito compartido puede incluir el alcance global o ámbitos de cierre. A menudo, en la programación orientada a objetos, los objetos se comparten entre ámbitos al agregar propiedades a otros objetos:

```
Materia materia = new Materia();
Usuario usuario = new Usuario();
usuario.materia = materia;
```

MUTABILIDAD

Un objeto inmutable es aquel que no puede ser modificado una vez haya sido creado. Por ejemplo, una variable en Java, no es un objeto inmutable.

```
String caracteres = "ABC";  
String nuevaCadana = caracteres.toLowerCase();
```

Podemos ver a los objeto inmutables como constantes, una vez nosotros asignamos un valor este no podrá ser modificado. La inmutabilidad también la podemos ver en este ejemplo: si queremos ordenar una lista de números enteros, lo que haremos será pasarle una función que nos devolverá una lista nueva pero ordenada sin tocar la lista original.

EFFECTOS SECUNDARIOS

Un efecto secundario es cualquier cambio de estado en la aplicación que sea observable fuera de la función llamada. Algo complejo, ¿no? Examinemos un poco nuestro código.

```
public static int funcion(int x){  
    variable_global = 10; //Cambio de valor  
    return x + 10;  
}
```

En este caso la "función" no es una función pura, ya que esta posee efectos secundarios al modificar el valor de una variable externa a la función. Los efectos secundarios se evitan principalmente en la programación funcional, para tener como resultado programas mucho más fáciles de comprender y de probar.

2.3 Expresiones lambda

Una expresión lambda, también llamada función lambda, función literal o función anónima, es una subrutina definida que no está enlazada a un identificador (es decir, no tiene nombre). Son muy comunes en los lenguajes que implementan programación funcional.

En Java se añadieron con la versión 8. Muy usadas en el manejo de Streams y en los eventos de interfaces gráficas. En el apartado anterior ya se han empleado pero ahora vamos a formalizarlas.

Una **expresión lambda** es un bloque corto de código que toma parámetros y devuelve un valor, similar a un método, pero sin necesidad de nombre. consiste en una lista separada por comas de los parámetros formales encerrados entre paréntesis, un símbolo de flecha (->) y un cuerpo. Los tipos de datos de los parámetros siempre se pueden omitir, al igual que los paréntesis si solo hay un parámetro. El cuerpo puede consistir en una sentencia o un bloque de sentencia.

Sintaxis: (parámetros) -> {cuerpo-lambda}

Cuando se tiene un solo parámetro no es necesario utilizar los paréntesis.

Cuando el cuerpo tiene una sola línea no hace falta usar las llaves ni poner return en caso de devolver valores.

Veamos algunos ejemplos

```
// sin parámetros
() -> System.out.println("Hello, world.")

// Con un parámetro (este ejemplo es una función de identidad).
a -> a

// Con una expresión
(a, b) -> a + b

// Con información de tipo explícita
(long id, String name) -> "id: " + id + ", name:" + name

// Con un bloque de código
(a, b) -> { return a + b; }

/* Con múltiples afirmaciones en el cuerpo lambda. Necesita un bloque de
código. Este ejemplo también incluye dos expresiones lambda anidadas (la
primera también es una clausura). */

(id, defaultPrice) -> {
    Optional<Product> product = productList.stream().filter(p -> p.getId()
== id).findFirst();
    return product.map(p -> p.getPrice()).orElse(defaultPrice);
}
```

Limitaciones en Java

Las expresiones lambda en Java 8 poseen las siguientes limitaciones:

- Pueden regresar excepciones comprobadas, pero tales lambdas no funcionarán con las interfaces utilizadas por la API de la colección.
- Las variables que están dentro del alcance donde se declara el lambda solo se pueden acceder dentro del lambda si son efectivamente finales, es decir, si la variable no está mutada dentro o fuera del ámbito lambda.

2.4 Programación declarativa en Java: uso de Streams

Gracias a la llegada de Java 8, y a la introducción de los streams, las colecciones han aumentado su funcionalidad. Los streams permiten realizar operaciones funcionales sobre los elementos de las colecciones.

A continuación, mostramos un ejemplo de las bondades de los streams donde, a partir de una lista de personas (cada una de ellas tiene un nombre), obtenemos una lista con todos los

nombres (en una sola línea de código, piensa en cómo lo harías sin usar “streams”):

```
List people = new ArrayList(); //Rellenarlo...  
List names = people.stream().map(Person::getName).collect(Collectors.toList());
```

Como hemos visto en el ejemplo, de una forma muy fácil hemos obtenido todos los nombres.

Veamos más ejemplos del uso de Streams

Supongamos que tenemos un ArrayList de objetos tipo “Persona” llamado “personas”. El siguiente ejemplo obtiene una estructura nueva llamada “adultos”, a partir de “personas” con las que cumplen la condición de tener una edad mayor o igual a 18 años. Además, las ordena por edad.

```
Stream<Persona> adultos =  
personas.stream()  
    .filter(p -> p.getEdad() >= 18)  
    .sorted((p1,p2) -> p1.getEdad() - p2.getEdad());
```

Este ejemplo obtiene una estructura nueva llamada “edades”, a partir de “personas” que contiene números enteros, concretamente las edades de cada persona. Sólo las edades.

```
Stream<Integer> edades =  
personas.stream()  
    .map(p -> p.getEdad());
```

Este ejemplo es similar al anterior, pero sólo almacena las edades que son mayor o igual a 18 años.

```
Stream<Integer> edadesadultos =  
personas.stream()  
    .filter(p -> p.getEdad() >= 18)  
    .map(p -> p.getEdad());
```

Estudia los siguientes 3 ejemplos y reflexiona, ¿qué consigue cada uno?

```
List<Integer> edadesadultos =  
personas.stream()  
    .filter(p -> p.getEdad() >= 18)  
    .map(p -> p.getEdad())  
    .collect(Collectors.toList());
```

```
personas.stream()  
    .filter(p -> p.getEdad() >= 18)  
    .map(p -> p.getEdad())  
    .forEach(p -> System.out.println(p));
```

```
double mediaedadadultos =  
    .filter(p -> p.getEdad() >= 18)  
    .map(p -> p.getEdad())  
    .mapToInt(p -> p.getEdad()).average().getAsDouble();
```

3. LENGUAJES MULTIPARADIGMA Y PROGRAMACIÓN CON PYTHON

Son aquellos que soportan la combinación de 2 o más paradigmas, permitiendo crear programas usando más de un estilo de programación. El objetivo en el diseño de estos lenguajes es permitir a los programadores utilizar el mejor paradigma para cada trabajo, admitiendo que ninguno resuelve todos los problemas de la forma más fácil y eficiente posible. Hoy en día la mayoría de lenguajes más usados son multiparadigma.

Ejemplos:

- C++: combina la programación estructurada con la orientación a objetos.
- Python/PHP: son orientados a objetos, reflexivos, estructurados y funcionales.
- Java: es fundamentalmente orientado a objetos ("casi" todo en Java es un objeto), incluye el imperativo y también el funcional desde la versión 8.
- Javascript: orientado a objetos (basado en prototipos), imperativo y con soporte para funcional.

3.1 Características de otros lenguajes multiparadigma con enfoque web

- **Python:** multipropósito.

En el caso web se usa fundamentalmente en backend (con Django). También existe una implementación para ser usado en frontend: Brython. Brython se ha diseñado para reemplazar a JavaScript como lenguaje de scripting para web. Por lo tanto, es una implementación de Python adaptada a un entorno HTML5, es decir, con una interfaz con los objetos y eventos DOM. Sin embargo, al ser un traductor de Python a Javascript, está algo limitado.

- **JavaScript:** sólo web (fundamentalmente frontend) e interpretado en la máquina cliente. NodeJS es la versión para backend. Tiene multitud de librerías y frameworks extras como: jQuery, React JS, AngularJS, Vue.js, etcétera...

Orientado a objetos, ya que maneja objetos dentro de la página Web sobre los cuales podemos definir diferentes eventos. Dichos objetos facilitan la programación de paginas interactivas a través del uso del DOM (Document Object Model). El DOM es esencialmente una interfaz que proporciona un conjunto estándar de objetos para representar documentos HTML, XHTML y XML, un modelo estándar sobre cómo pueden combinarse dichos objetos, y una interfaz estándar para acceder a ellos y manipularlos. A través del DOM, los programas pueden acceder y modificar el contenido, estructura y estilo de los documentos HTML y XML, que es para lo que se diseñó principalmente.

Responde a eventos en tiempo real. Eventos como presionar un botón, pasar el puntero del ratón sobre un determinado texto o el simple hecho de cargar la página o caducar un tiempo. Con esto podemos cambiar totalmente el aspecto de la página al gusto del usuario, evitándonos tener en el servidor un página para cada

gusto, hacer cálculos en base a variables cuyo valor es determinado por el usuario, etc.

- **PHP:** sólo web (backend) y ejecutado en la máquina servidor por un intérprete, la cual genera como resultado una web que se envía al cliente.

Es un lenguaje muy extendido y de larga tradición en el ámbito web. Es un lenguaje de alto nivel, orientado a objetos y básicamente imperativo, aunque incluye un pequeño soporte al paradigma declarativo estilo funcional. Permite el uso de patrones de diseño, como el MVC (Modelo Vista Controlador).

3.2 El tipado de datos

El tipado es una característica importante y, al hablar de lenguajes multiparadigma, debemos detallarlo. El tipado, o sistema de tipos, define la forma en que un lenguaje de programación clasifica los valores y las expresiones, definiendo cómo se pueden manipular estos tipos y cómo interactúan. Un tipo de dato indica un conjunto de valores que tienen el mismo significado genérico o propósito (aunque algunos tipos, como los tipos de datos abstractos y tipos de datos función tal vez no representen valores en el programa que se está ejecutando). Los sistemas de tipificación varían significativamente entre lenguajes, siendo quizás las más importantes variaciones las que estén en sus propias implementaciones de la sintáctica en tiempo de compilación y la operativa en tiempo de ejecución.

Hay 2 clasificaciones fundamentales: fuerte-débil, estático-dinámico; y sus combinaciones. Veamoslas:

Tipado fuerte

- Java
- C#
- Python

Un lenguaje es fuertemente tipado si no se permiten violaciones de los tipos de datos, es decir, dado el valor de una variable de un tipo concreto, no se puede usar como si fuera de otro tipo distinto a menos que se haga una conversión. Cuando un lenguaje es fuertemente tipado, el programador debe explícitamente convertir un tipo de datos en otro. Esto se conoce como conversión o moldeado de tipos (typecasting) y también debe estar soportado por el compilador.

Tipado débil

- JavaScript
- PHP

Los lenguajes de programación no tipados o débilmente tipados no controlan los

tipos de las variables que declaran, de este modo, es posible usar variables de cualquier tipo en un mismo escenario. Por ejemplo, una función puede recibir como parámetro un valor entero, cadena de caracteres, flotante, etc. No hay que confundir el término con los lenguajes de tipos dinámicos, en los que los tipos de las variables se deciden en tiempo de ejecución, si bien es cierto que muchos lenguajes de programación de tipos dinámicos (en los que no se declaran los tipos de datos) son también no tipados.

No son necesarias las conversiones de tipos, por lo tanto el siguiente ejemplo Javascript es correcto:

```
var a=3;
var b="4";
var c=a+b; // c contiene la cadena "34"
```

Tipado estático

Se dice que lenguaje de programación usa un tipado estático cuando la comprobación de tipos se realiza durante la compilación, y no durante la ejecución. Ejemplos de lenguajes que usan tipado estático son: C, C++, Java y Haskell. Comparado con el tipado dinámico, el estático permite que los errores de tipificación sean detectados antes, y que la ejecución del programa sea más eficiente y segura.

```
int var; //declara una variable que sólo acepta números enteros
```

Tipado dinámico

Los tipos de las variables se deciden en tiempo de ejecución. Por tanto, una variable puede tomar valores de distintos tipos. La mayoría de lenguajes de tipado dinámico son lenguajes interpretados, como Python o Ruby. Un lenguaje que no es dinámicamente tipado se dice que es de tipado estático, o estáticamente tipado. Esta separación se suele confundir con la diferencia entre lenguajes fuertemente tipados y lenguajes débilmente tipados.

```
var = 3; #Aquí la variable "var" adopta el tipo de datos entero
var = 2.3; # Aquí se adapta al tipo de datos decimal o coma flotante
```

Veamos cómo se clasifican algunos de los lenguajes más conocidos:

- Java, C#: Tipado fuerte y estático.
- Python: Tipado fuerte y dinámico.
- Javascript / PHP: Tipado débil y dinámico.

3.3 El lenguaje de programación Python

Python es un lenguaje de programación de propósito general muy poderoso y flexible, a la vez que sencillo y fácil de aprender. Es un lenguaje de alto nivel, que permite procesar

fácilmente todo tipo de estructuras de datos, tanto numéricos como de texto. Este lenguaje fue creado a principios de los noventa por Guido van Rossum en los Países Bajos. En este apartado se busca introducir el lenguaje Python, sus características y los recursos mas destacados disponibles en la Web para tomar en cuenta; de forma muy resumida.

3.3.1 Características de Python

- Es un lenguaje **interpretado**, de alto nivel (muy alto nivel), no compilado, usa **tipado dinámico** y es **fuertemente tipado**.
- Es **multiplataforma**, lo cual es ventajoso para hacer ejecutable su código fuente entre varios sistema operativos.
- Es un lenguaje de programación **multiparadigma**, el cual soporta varios paradigma de programación como orientación a objetos, estructurada, programación imperativa y, en menor medida, programación funcional.
- En Python, el formato del código (p. ej., la indentación) es **estructural**.
- Sintaxis limpia y fácil de aprender.
- Muchos módulos y librerías de soporte para cualquier ámbito (web, Big Data, etc.)
- Usado y demandado por grandes empresas (Google, IBM, etc...)

El **fuertemente tipado** significa que el tipo de valor no cambia repentinamente. Un string que contiene solo dígitos no se convierte mágicamente en un número. Cada cambio de tipo requiere una conversión explícita. A continuación un ejemplo de este concepto:

```
# se usa el metodo int() para convertir a entero
total = valor1 + int(valor2)
# se usa el metodo str() para convertir a cadena
print ("El total es: " + str(total))
```

El **tipado dinámico** significa que los objetos en tiempo de ejecución (valores) tienen un tipo, a diferencia del tipado estático donde las variables tienen un tipo. A continuación un ejemplo de este concepto:

```
print (variable, type(variable))
# "variable" guarda un valor string
variable = "activo"
print (variable, type(variable))
```

3.3.2 Variables y tipos de datos

Para utilizar una variable en python simplemente hay que asignarle un valor, por ejemplo:

```
nombre = 'Jose'
```

Esto creará una variable llamada nombre, a la que se le asignará el valor "Joe", las variables que contienen cadenas de caracteres (a partir de ahora "strings") deben estar entre "," o "'", no pasa lo mismo con las variables que almacenan números, por ejemplo:

```
i = 10
```

El tipado dinámico asignará, automáticamente, y en tiempo de ejecución, el tipo "entero" a la variable "i", de la misma forma que asignará el tipo "cadena (string)" a la variable "nombre" anterior. Prueba a sumar ambas...¿qué sucede?, ¿porqué?

Se pueden reutilizar variables y su tipo de datos cambiará automáticamente, por ejemplo:

```
i = 200 [tipo entero]
```

```
i = 'xyz' [tipo string]
```

Las reglas de nombrado son las mismas que en la mayoría de lenguajes. Por otra parte, y como podemos observar, los comandos terminan con salto de línea, a diferencia de otros lenguajes, como Java o C++ que deben terminar con ";

En python existen varios tipos de datos, a continuación los más destacados:

- A) **Números** (enteros o decimales)
- B) **Strings** (cadenas de caracteres)
- C) **Caracteres individuales**
- D) **Booleanos** (true, false)
- E) **Listas** (incluye el concepto de array) y **tuplas** (cadenas **NO modificables**)
- F) **Diccionarios** (equivalente a las tablas hash)

3.3.3 Variables numéricas

Los números se operan como se harían naturalmente, enteros y flotantes pueden operar entre ellos sin ningún problema, pero dado que operar dos enteros da como resultado un entero, (si al menos uno de los dos es flotante el resultado también lo será), esto se nota especialmente en las divisiones:

```
>>> 10/3
3
>>> 10/3. # Un 3 de punto flotante 3.3333333333333335
>>> 1+1
2
>>> 1+1.
2.0
>>> 2/2 1
>>> 1/2
0
>>> 1/2. 0.5
>>> 2*5
10
```

```
>>> 1-2
-1
```

3.3.4 Cadenas de caracteres (string)

Los strings son cadenas de caracteres, para pasar una variable cualquiera a string se utiliza:

str(variable)

Útil para, por ejemplo, concatenar variables numéricas con textos para mostrarlos en un print.

Las cadenas se “comportan” automáticamente como un array de caracteres, por lo que podemos hacer esto:

```
>>> cadena = "josé"
>>> cadena[0]
'j'
>>> cadena[3]
'é'
# Operador "slice", ojo al dato, esto es genial en Python
>>> cadena[0:2]
'jos'
```

3.3.5 Entrada y salida básica

Para interactuar con el usuario podemos hacerlo mediante:

- **input:** leer de teclado (Como Scanner y luego nextLine....) mostrando a la vez el mensaje. Ojo, input siempre lee en formato string, si queremos procesar un número captado por teclado hay que transformarlo, ejemplo:

```
edad = int(input('Introduce tu edad:'))
```

- **print:** mostrar en pantalla (Como System.out.println de Java)

```
print('Tu edad es: ',edad)
```

3.3.6 Estructuras de control (condicionales y bucles)

Para decidir si una parte del programa se ejecuta o no se utiliza las funciones if (si), elif (o-si) y else (sino)

Ejemplo:

```
numero = input('Cuanto es 1+1? ')

if (len(numero)==0):
    print('No has introducido nada')

elif(numero!='2'):
    print('No, te has equivocado')

else:
```

```
print('Si, has acertado')
```

Cuidado con las identaciones y el símbolo “:” al final de cada instrucción condicional...

Operadores básicos

- Operadores aritméticos: +, -, *, /, //, %, **.
- Operadores de cadenas: +, *
- Operadores de asignación: =
- Operadores de comparación: ==, !=, >=, >, <=, <
- Operadores lógicos: and, or, not
- Operadores de pertenencia: in, not in

Hay dos tipos de bucles: bucles while y bucles for

Los bucles while ejecutan un código mientras se dé la condición, por ejemplo:

```
i = 10
while i>=0:
    print(i)
    i-=1
print ('Despegue!')
```

El bucle do-while, existente en otros lenguajes, no existe como tal en Python por lo que hay que “simularlo”.

En el siguiente código, intentamos emular un bucle “do-while” para imprimir valores del 1 al 10.

```
x = 0
while True:
    print(x)
    x = x+1
    if(x>10):
        break
```

En el método anterior ponemos la condición como “True” para que el bucle while se ejecute al menos una vez, y más adelante en el bucle, probamos la condición para detener el bucle. La declaración “break” aquí se utiliza para salir del bucle tan pronto como se cumple la condición deseada. Podemos utilizar el concepto de variable “bandera” para evitar el “break”.

Los bucles for, ejecutan el código por cada elemento en una lista (o string), por ejemplo:

```
for i in range(0,15):
    print('Valor de i: ' + str(i))
```


Este bucle imprime en pantalla la frase seguida del número 0 hasta el 14.

En Python, el bucle for es muy potente, veamos otros ejemplos:

```
# Mostrar todas las letras de una # palabra
palabra = input('Introduce una palabra: ')

for letra in palabra:
    print ('Letra: ', letra)
```

Acabas de presenciar un comportamiento automático de for-each.

3.3.7 Estructuras de datos

En Python se simplifican bastante las estructuras de datos, con respecto a otros lenguajes, vamos a ver algunas similitudes con Java:

- Listas (Python): arrays estáticos y dinámicos (ArrayList en Java).
- Diccionarios (Python): tabla hash (HashMap en Java).

LISTAS

```
lista = [1,2,3,4,5,6]
>>> lista[3]
4
>>> lista[12]
...
IndexError: list index out of range
>>> lista[-1]
6
>>> 2 in lista
True
>>> lista + [7,8,9]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> lista * 2
[1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6]
>>> lista[2:4]
[3, 4]
>>> lista[1:4:2]
[2, 4]
>>> lista[:5]
[1, 2, 3, 4, 5]
>>> lista[5:]
[6, 1, 2, 3, 4, 5, 6]
>>> lista[::-1]
[6, 5, 4, 3, 2, 1, 6, 5, 4, 3, 2, 1]
```

También multidimensionales

```
>>> tabla = [[1,2,3],[4,5,6],[7,8,9]]
>>> tabla[1][1]
5
>>> for fila in tabla:
...     for elem in fila:
...         print(elem,end="")
...     print()
```

DICCIONARIOS

```
>>> diccionario = {'one': 1, 'two': 2, 'three': 3}
>>> dict1 = {}
>>> dict1["one"]=1
>>> dict1["two"]=2
>>> dict1["three"]=3
>>> dict1 = {'one': 1, 'two': 2, 'three': 3}
>>> dict1.get("one")
1
```

Todas las estructuras de datos comentadas son objetos de alto nivel, y, como tales, poseen multitud de métodos que pueden ser llamados para realizar miles de operaciones sobre ellos. Como: append, pop, len, insert, remove, upper, lower, max, sort, reverse, index, count, y un largo etcétera...

Otros aspectos muy importantes en Python, pero que por necesidad de resumir y ser breves no vamos a detallar, invitando al lector a profundizar en ellos, son:

- Programación orientada a objetos
- Excepciones
- Carga y uso de módulos (como el import de Java)
- Funciones y métodos